Documentación de pruebas

Registro de 11 evidencias de ejecución del sistema cliente-servidor TCP y de las mejoras incorporadas a partir de las pruebas.

Entorno: Windows, Visual Studio Code, PowerShell y Python. Servidor TCP local: 127.0.0.1:12345. Procesamiento concurrente con un pool de 4 workers.

Organización: las evidencias 01 a 07 muestran las pruebas manuales iniciales; las evidencias 08 y 09, las pruebas automáticas y la concurrencia; las evidencias 10 y 11, las correcciones verificadas.

Lectura de las evidencias: cada caso presenta el procedimiento, el resultado observado y la captura de la terminal. Las pruebas automáticas incluyen una solicitud inválida intencional para verificar su rechazo.

Alcance: el diagrama Mermaid describe la arquitectura distribuida propuesta (balanceador, RabbitMQ, workers y almacenamiento). Las capturas de este documento corresponden al prototipo local ejecutado en Python; no prueban el despliegue de esos servicios externos.

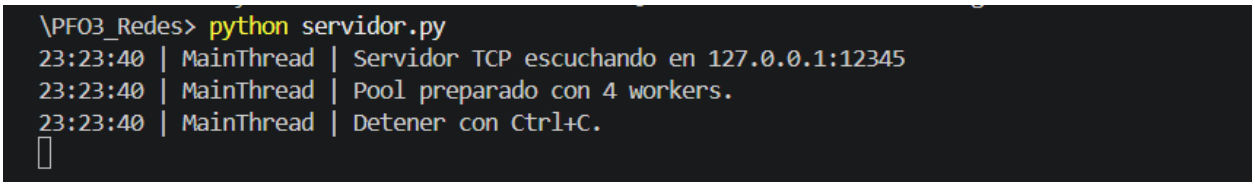
01. Inicio del servidor TCP.
02. Inicio del cliente de consola.
03. Suma de números.
04. Conversión de texto a mayúsculas.
05. Conteo de palabras: resultado inicial.
06. Inversión de caracteres: denominación inicial.
07. Registro de operaciones en el servidor.
08. Pruebas automáticas y concurrencia.
09. Distribución de tareas entre workers.
10. Conteo de palabras corregido.
11. Nombre de operación mejorado.

01. Servidor iniciado

Evidencia 01 | Servidor TCP / inicio del pool

Procedimiento: Ejecutar python servidor.py desde la carpeta PFO3_Redes.

Resultado observado: El servidor queda escuchando en 127.0.0.1:12345 e informa que el pool dispone de cuatro workers. Permanece activo a la espera de solicitudes.



Archivo: 01_servidor_iniciado.png

02. Cliente iniciado

Evidencia 02 | Menú de consola

Procedimiento: Mantener el servidor activo y ejecutar python cliente.py en otra terminal.

Resultado observado: Se despliega el menú con las opciones de sumar números, convertir texto a mayúsculas, contar palabras, invertir texto y salir. La aplicación espera una elección.



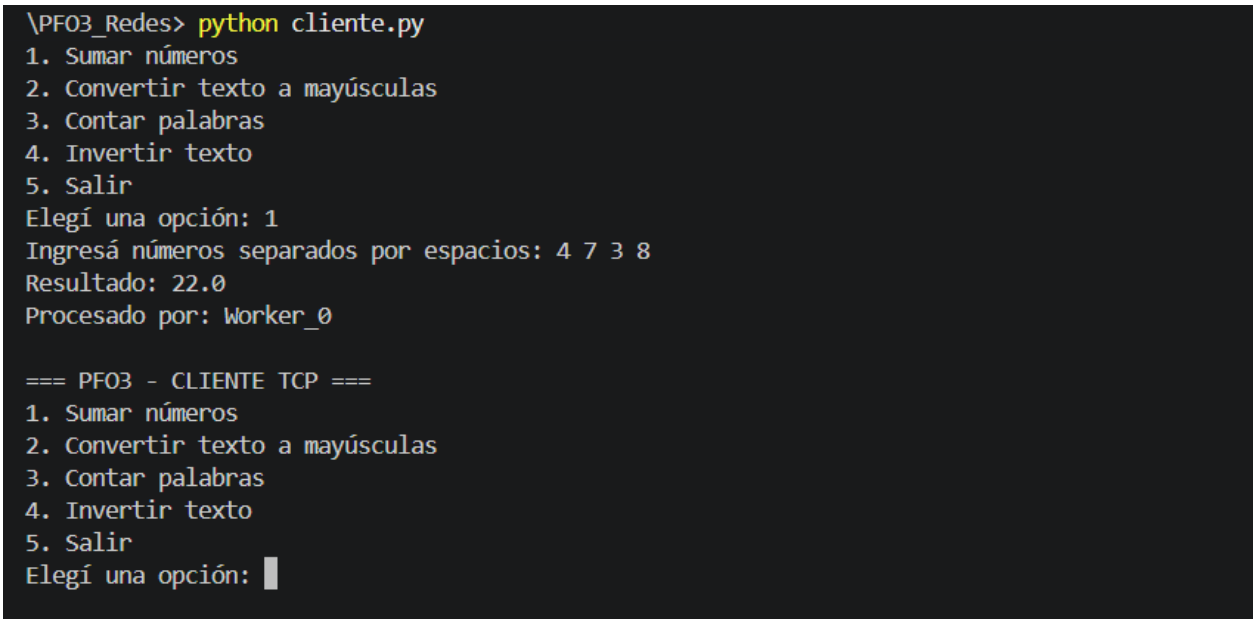
Archivo: 02_cliente_iniciado.png

03. Suma de números

Evidencia 03 | Opción 1 | Operación sumar

Procedimiento: Elegir 1 e ingresar los números 4 7 3 8, separados por espacios.

Resultado observado: El cliente recibe el resultado 22.0. La tarea fue procesada por Worker_0 y el menú vuelve a estar disponible.



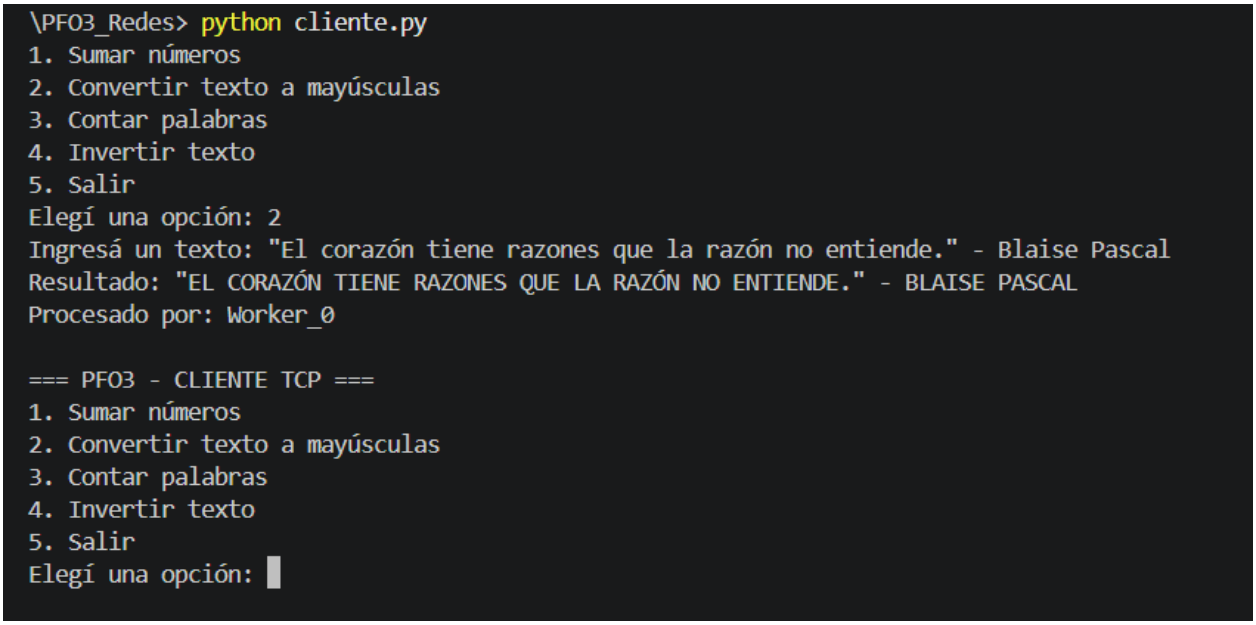
Archivo: 03_cliente_suma.png

04. Texto convertido a mayúsculas

Evidencia 04 | Opción 2 | Operación mayusculas

Procedimiento: Elegir 2 e ingresar la frase «El corazón tiene razones que la razón no entiende.» atribuida a Blaise Pascal, con la referencia escrita en la prueba.

Resultado observado: El texto se devuelve en mayúsculas, conservando correctamente los caracteres acentuados. Procesó la solicitud Worker_0.



Archivo: 04_cliente_mayusculas.png

05. Conteo de palabras: versión inicial

Evidencia 05 | Opción 3 | Conteo inicial

Procedimiento: Elegir 3 e ingresar la oración sobre Charles Babbage y Ada Lovelace que contiene el año 1843.

Resultado observado: El resultado es 19 porque el conteo inicial considera como elemento también el número 1843. La comunicación TCP y el procesamiento son correctos, pero se detecta una limitación funcional.

Observación: Este caso origina una mejora: excluir del conteo los elementos que no contienen letras. La validación posterior aparece en la evidencia 10.

```
\PFO3_Redes> python cliente.py
1. Sumar números
2. Convertir texto a mayúsculas
3. Contar palabras
4. Invertir texto
5. Salir
Elegí una opción: 3
Ingresá un texto: Charles Babbage diseñó la Máquina Analítica y Ada Lovelace publicó en 1843
 un algoritmo para calcular números de Bernoulli.
Resultado: 19
Procesado por: Worker_0

=== PFO3 - CLIENTE TCP ===
1. Sumar números
2. Convertir texto a mayúsculas
3. Contar palabras
4. Invertir texto
5. Salir
Elegí una opción:
```

Archivo: 05_cliente_contar_palabras.png

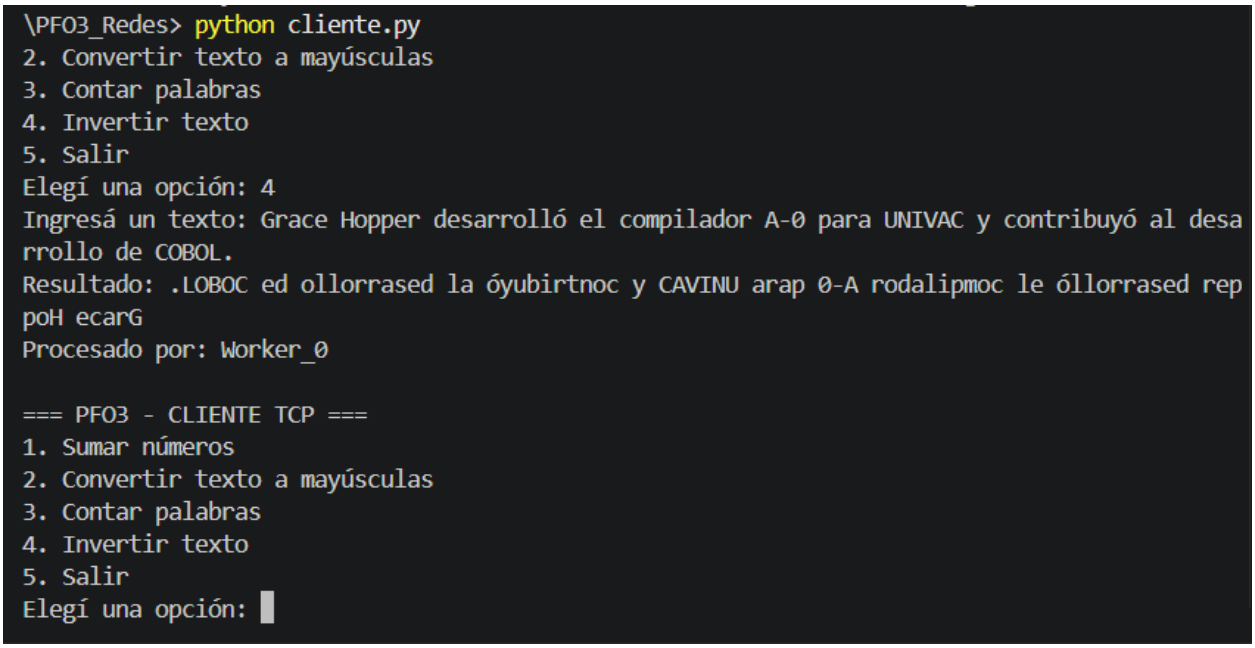
06. Inversión de caracteres: versión inicial

Evidencia 06 | Opción 4 | Denominación original

Procedimiento: Elegir 4 («Invertir texto») e ingresar la frase de Grace Hopper sobre el compilador A-0, UNIVAC y COBOL.

Resultado observado: El servidor devuelve los caracteres de la oración en orden inverso. La operación funciona, pero el nombre inicial puede confundirse con un reflejo visual en espejo.

Observación: Se decide reemplazar el rótulo del menú por «Invertir orden de caracteres», sin cambiar la operación interna invertir. La nueva verificación consta en la evidencia 11.



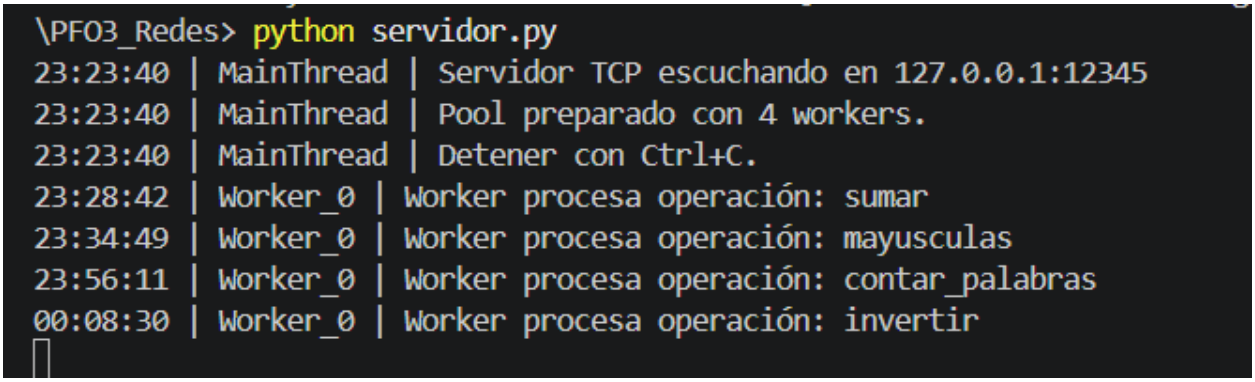
Archivo: 06_cliente_invertir_texto.png

07. Operaciones recibidas por el servidor

Evidencia 07 | Registro de las cuatro solicitudes manuales

Procedimiento: Consultar la terminal donde permanece activo servidor.py después de las cuatro operaciones ejecutadas desde el cliente.

Resultado observado: El registro muestra sumar, mayusculas, contar_palabras e invertir. Todas aparecen atendidas por Worker_0, algo compatible con el envío de solicitudes una por vez.



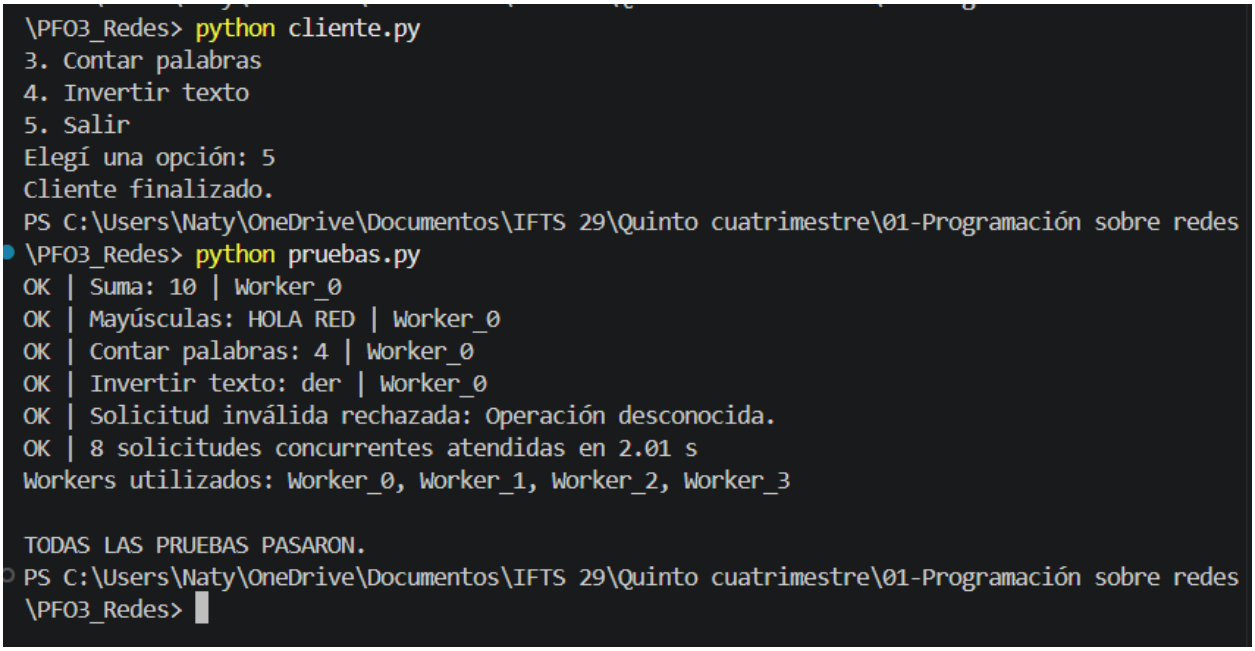
Archivo: 07_servidor_operaciones.png

08. Pruebas automáticas y concurrencia

Evidencia 08 | Ejecución de pruebas.py

Procedimiento: Cerrar el cliente con la opción 5, mantener activo el servidor y ejecutar python pruebas.py en la terminal.

Resultado observado: Se validan las cuatro operaciones, el rechazo de una operación desconocida y la atención de ocho solicitudes concurrentes. La salida indica 2.01 s, uso de Worker_0 a Worker_3 y «TODAS LAS PRUEBAS PASARON».



Archivo: 08_workers_concurrentes.png

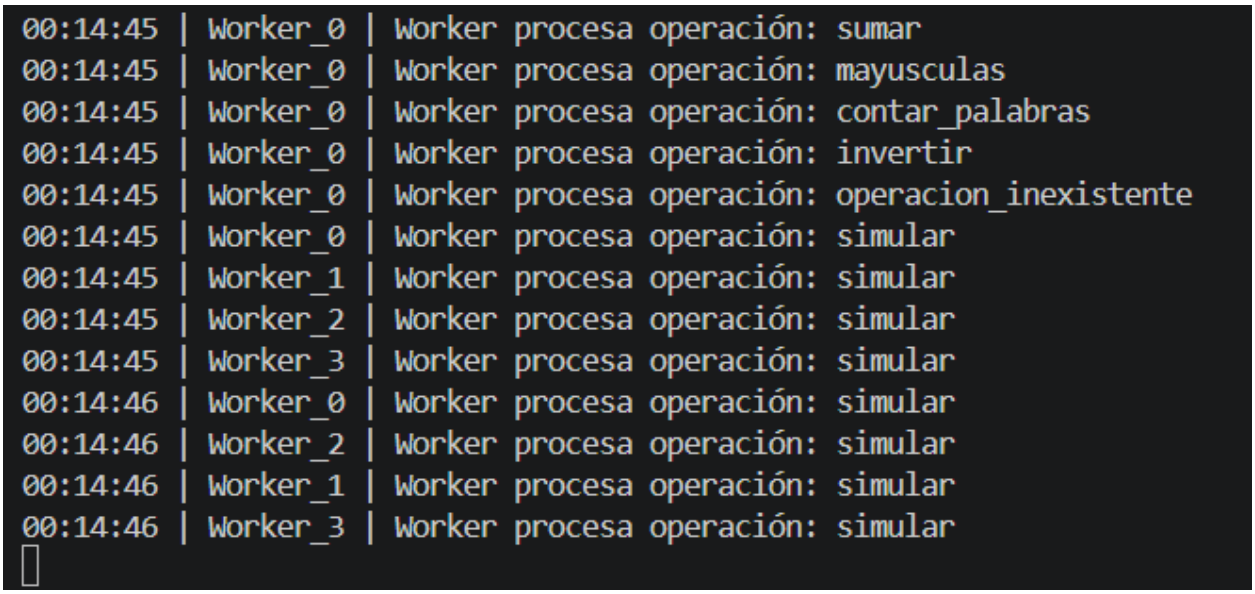
09. Distribución de tareas entre workers

Evidencia 09 | Registro del servidor durante las pruebas automáticas

Procedimiento: Observar los mensajes registrados por servidor.py durante la ejecución de pruebas.py.

Resultado observado: El servidor registra las operaciones de prueba y las solicitudes simular. Se identifican los cuatro trabajadores, Worker_0, Worker_1, Worker_2 y Worker_3, realizando tareas.

Observación: La operación operacion_inexistente se envió deliberadamente para comprobar la respuesta ante una solicitud inválida.



Archivo: 09_servidor_workers.png

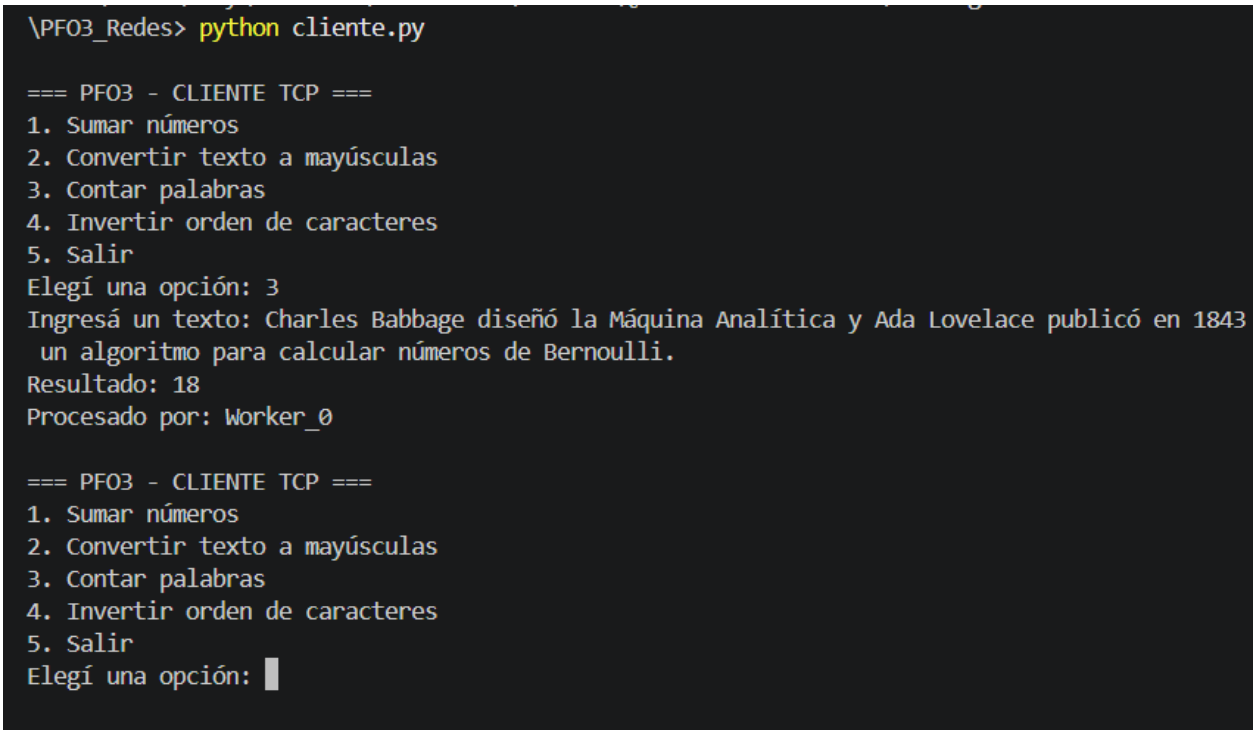
10. Conteo de palabras corregido

Evidencia 10 | Opción 3 | Verificación después del cambio

Procedimiento: Actualizar en servidor.py la lógica del conteo para considerar únicamente elementos que contengan letras, reiniciar el servidor y repetir la misma frase con 1843.

Resultado observado: La respuesta pasa de 19 a 18 palabras. El año 1843 deja de incluirse en el conteo y se conserva la respuesta correcta del servidor al cliente.

Observación: La corrección se adapta al caso probado; no constituye un analizador lingüístico completo para todos los formatos posibles de texto.



Archivo: 10_conteo_corregido.png

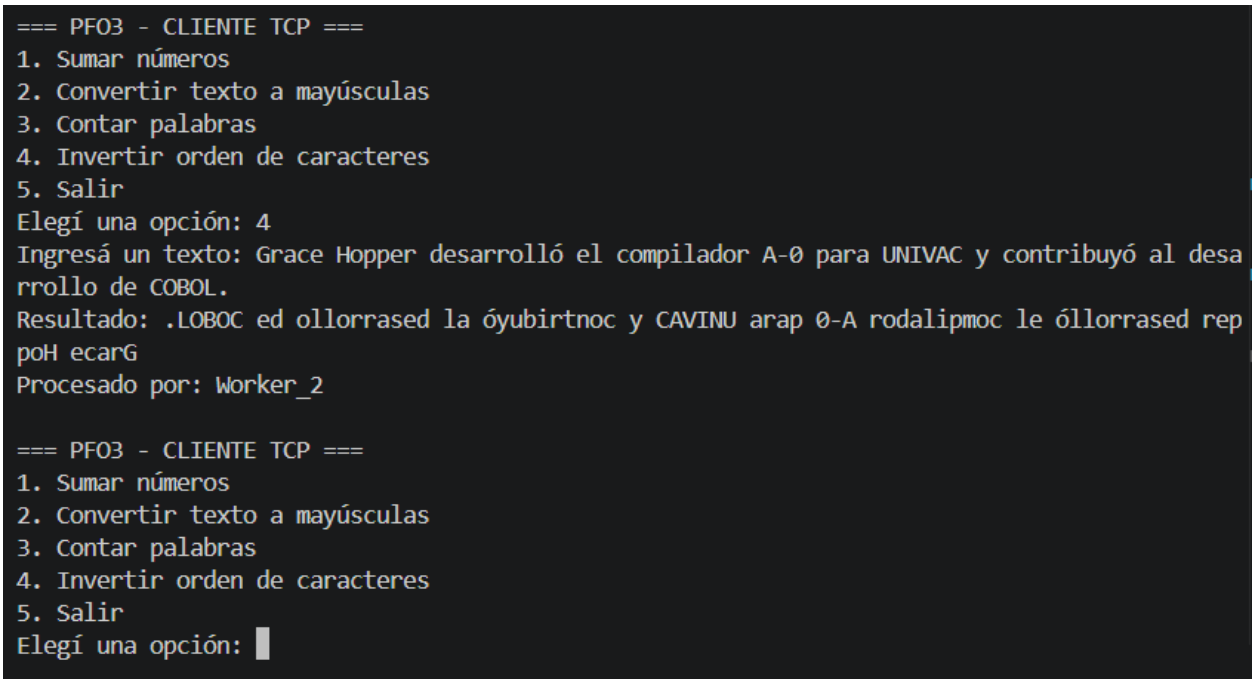
11. Denominación de opción mejorada

Evidencia 11 | Opción 4 | Invertir orden de caracteres

Procedimiento: Modificar en cliente.py solo el texto visible de la opción 4; reiniciar el cliente e ingresar nuevamente la frase sobre Grace Hopper.

Resultado observado: El menú indica «Invertir orden de caracteres». La inversión continúa funcionando y en esta ejecución el resultado lo procesó Worker_2.

Observación: El identificador de la operación que envía el cliente al servidor continúa siendo invertir.



Archivo: 11_nombre_operacion_corregido.png

Registro de mejoras y conclusión

Las pruebas permitieron identificar dos oportunidades de mejora y comprobar los resultados después de modificar el código.

Mejora 1. Conteo de palabras

Antes: 19 elementos, porque el número 1843 se contaba junto con las palabras. Después: 18 palabras, al excluir el elemento numérico. Evidencias relacionadas: 05 y 10.

Mejora 2. Nombre de la operación

Antes: el menú mostraba «Invertir texto». Después: se muestra «Invertir orden de caracteres», sin modificar la función que revierte la secuencia. Evidencias relacionadas: 06 y 11.

Resultado de la verificación

El cliente se conectó al servidor TCP, envió operaciones y recibió resultados. Las pruebas automáticas iniciales verificaron también el rechazo de una solicitud desconocida y la distribución de ocho solicitudes concurrentes entre cuatro workers. Tras los cambios, la ejecución de pruebas.py volvió a finalizar satisfactoriamente, según la comprobación realizada en la terminal.

Alcance del trabajo

Estas pruebas documentan el funcionamiento local del cliente, el servidor y el pool de hilos. El balanceador Nginx/HAProxy, RabbitMQ, PostgreSQL y S3 están representados en el diagrama de arquitectura distribuida como componentes propuestos, no como servicios desplegados y verificados en este prototipo.

Archivos de referencia: servidor.py, cliente.py, pruebas.py y docs/diagrama_arquitectura.png.